

Authentication System

Complete guide to Gema's authentication and authorization system.

Overview

Gema uses a dual authentication system:

- **JWT Tokens:** For API access and session management
- **Firebase Auth:** For social login and identity verification

Quick Start

Basic Authentication Flow

```
# 1. Register new user
curl -X POST http://localhost:5000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "password": "SecurePass123!"
  }'

# 2. Login and get tokens
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "john@example.com",
    "password": "SecurePass123!"
  }'

# 3. Use access token for protected endpoints
curl -X GET http://localhost:5000/api/auth/me \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

User Registration & Login

POST /api/auth/register

Register a new user account.

Request Body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "password": "SecurePassword123!",
  "phone": "+1234567890",
  "country": "UAE"
}
```

Password Requirements:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character (!@#\$\$%^&*)

Response:

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user": {
      "id": "user_id_here",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john.doe@example.com",
      "role": "customer",
      "isEmailVerified": false,
      "status": "PENDING"
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}
```

POST /api/auth/login

Login with email and password.

Request Body:

```
{
  "email": "john.doe@example.com",
  "password": "SecurePassword123!"
}
```

Response:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "user": {
      "id": "user_id_here",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john.doe@example.com",
      "role": "customer"
    },
    "tokens": {
      "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    }
  }
}
```

Token Management

Access Tokens

- **Lifetime:** 7 days (configurable)
- **Usage:** Include in Authorization: Bearer <token> header
- **Payload:** Contains user ID and role information

Refresh Tokens

- **Lifetime:** 30 days (configurable)
- **Usage:** Obtain new access tokens when they expire
- **Storage:** Stored in database with metadata (IP, user agent)

POST /api/auth/refresh-token

Get new access token using refresh token.

Request Body:

```
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response:

```
{
  "success": true,
  "message": "Token refreshed successfully",
  "data": {
    "user": {
      "id": "user_id",
      "firstName": "John",
      "lastName": "Doe",
      "role": "customer"
    },
    "tokens": {
      "accessToken": "new_access_token",
      "refreshToken": "new_refresh_token"
    }
  }
}
```

POST /api/auth/logout

Logout and invalidate tokens.

Headers:

```
Authorization: Bearer <access_token>
```

Optional Body:

```
{
  "refreshToken": "refresh_token_to_revoke"
}
```

 Firebase Authentication

For social login and React Native integration.

POST /api/auth/firebase

Authenticate using Firebase ID token.

Request Body:

```
{
  "idToken": "firebase_id_token_from_client"
}
```

Response:

```
{
  "success": true,
  "message": "Firebase authentication successful",
  "data": {
    "user": {
      "id": "user_id",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com",
      "role": "customer",
      "isEmailVerified": true
    },
    "tokens": {
      "accessToken": "backend_jwt_token",
      "refreshToken": "backend_refresh_token"
    }
  }
}
```

Firebase Integration Flow

1. Client authenticates with Firebase

```
// React Native / Web
const result = await auth().signInWithEmailAndPassword(email, password);
const idToken = await result.user.getIdToken();
```

2. Send ID token to backend

```
const response = await fetch('/api/auth/firebase', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ idToken })
});
```

3. Backend verifies and creates session

- Verifies Firebase ID token
- Creates/finds user in MongoDB
- Issues backend JWT tokens
- Returns session information

✉ Email Verification

OTP-Based System

- **4-digit numeric codes** for mobile-friendly experience
- **10-minute expiration** for security
- **Professional email templates**
- **Resend functionality**

POST /api/auth/verify-email

Verify email using OTP code.

Request Body:

```
{
  "otp": "1234"
}
```

Response:

```
{
  "success": true,
  "message": "Email verified successfully"
}
```

POST /api/auth/resend-verification-email

Request new verification OTP.

Request Body:

```
{  
  "email": "john@example.com"  
}
```

Password Management

POST /api/auth/forgot-password

Request password reset email.

Request Body:

```
{  
  "email": "john@example.com"  
}
```

Response:

```
{  
  "success": true,  
  "message": "If your email is registered, you will receive a password reset link"  
}
```

POST /api/auth/reset-password

Reset password using reset token.

Request Body:

```
{  
  "token": "password_reset_token_from_email",  
  "newPassword": "NewSecurePassword123!"  
}
```

PUT /api/auth/change-password

Change password (requires authentication).

Headers:

```
Authorization: Bearer <access_token>
```

Request Body:

```
{  
  "currentPassword": "CurrentPassword123!",  
  "newPassword": "NewSecurePassword123!"  
}
```

User Profile Management

GET /api/auth/me

Get current authenticated user profile.

Headers:

```
Authorization: Bearer <access_token>
```

Response:

```
{
  "success": true,
  "message": "User profile retrieved successfully",
  "data": {
    "user": {
      "_id": "user_id",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com",
      "role": "customer",
      "phone": "+1234567890",
      "country": "UAE",
      "avatar": "https://cloudinary.com/avatar.jpg",
      "isEmailVerified": true,
      "status": "ACTIVE",
      "addresses": [],
      "createdAt": "2024-01-15T10:00:00Z",
      "lastLogin": "2024-03-16T14:30:00Z"
    }
  }
}
```

PUT /api/auth/profile

Update user profile information.

Headers:

```
Authorization: Bearer <access_token>
```

Request Body:

```
{
  "firstName": "John",
  "lastName": "Smith",
  "phone": "+1234567890",
  "country": "UAE",
  "gender": "male",
  "dateOfBirth": "1990-01-15"
}
```

 Authorization & Roles

User Roles

- **customer:** Can browse events, make orders, write reviews
- **vendor:** Can create/manage events, view analytics, manage employees
- **admin:** Full system access, user management, content moderation
- **employee:** Limited access based on assigned permissions

Role-Based Access Control

Usage in routes:

```
// Require authentication only
router.use(authenticate);

// Require specific roles
router.use(authorize(['admin', 'vendor']));

// Single role
router.use(authorize(['customer']));
```

Permission Hierarchy:

```
Admin > Vendor > Employee > Customer
```

Protected Route Examples

```
# Customer only
POST /api/orders
GET /api/tickets

# Vendor only
POST /api/events
GET /api/events/vendor/my-events

# Admin only
GET /api/admin/users
PUT /api/admin/events/:id/approval

# Multiple roles
GET /api/analytics/dashboard # Admin or Vendor
```

Security Features

Password Security

- **Bcrypt** hashing with salt rounds
- **Password strength requirements**
- **Rate limiting** on auth endpoints

Session Security

- **JWT token validation**
- **Token expiration handling**
- **Refresh token rotation**
- **IP and user agent tracking**

Request Security

- **CORS protection** configured
- **Rate limiting** (100 requests per 15 minutes)
- **Input validation** with express-validator
- **SQL injection protection** via Mongoose
- **XSS protection** via helmet

Implementation Examples

Frontend Token Storage

```
// Store tokens securely
localStorage.setItem('accessToken', data.tokens.accessToken);
localStorage.setItem('refreshToken', data.tokens.refreshToken);

// Add to API requests
const token = localStorage.getItem('accessToken');
fetch('/api/protected-endpoint', {
  headers: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  }
});
```

Auto Token Refresh

```
// Interceptor for automatic token refresh
axios.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      const refreshToken = localStorage.getItem('refreshToken');
      try {
        const response = await axios.post('/api/auth/refresh-token', {
          refreshToken
        });
        const newToken = response.data.data.tokens.accessToken;
        localStorage.setItem('accessToken', newToken);

        // Retry original request
        error.config.headers.Authorization = `Bearer ${newToken}`;
        return axios.request(error.config);
      } catch (refreshError) {
        // Redirect to login
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);
```

Error Handling

```
// Handle common auth errors
const handleAuthError = (error) => {
  switch (error.response?.status) {
    case 401:
      // Token expired or invalid
      redirectToLogin();
      break;
    case 403:
      // Insufficient permissions
      showError('Access denied');
      break;
    case 429:
      // Rate limited
      showError('Too many requests, please try again later');
      break;
    default:
      showError('Authentication error occurred');
  }
};
```

Testing Authentication

Test Users (from seeded data)

```
# Admin user
curl -X POST http://localhost:5000/api/auth/login \
  -d '{"email":"admin@ge.ma.com","password":"admin123"}'

# Vendor user
curl -X POST http://localhost:5000/api/auth/login \
  -d '{"email":"vendor@ge.ma.com","password":"vendor123"}'

# Customer user
curl -X POST http://localhost:5000/api/auth/login \
  -d '{"email":"customer@ge.ma.com","password":"customer123"}'
```

Next Steps:

- [Complete API Reference → \(./api-reference.md\)](#) - Detailed endpoints
- [Getting Started → \(./architecture/getting-started.md\)](#) - Complete setup guide